

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline
```

```
df = pd.read_csv("/content/height-weight.csv")
```

```
df.head()
```

	Weight	Height
0	45	120
1	58	135
2	48	123
3	60	145
4	70	160

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23 entries, 0 to 22
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Weight   23 non-null        int64
1   Height   23 non-null        int64
dtypes: int64(2)
memory usage: 500.0 bytes
```

```
df.shape
```

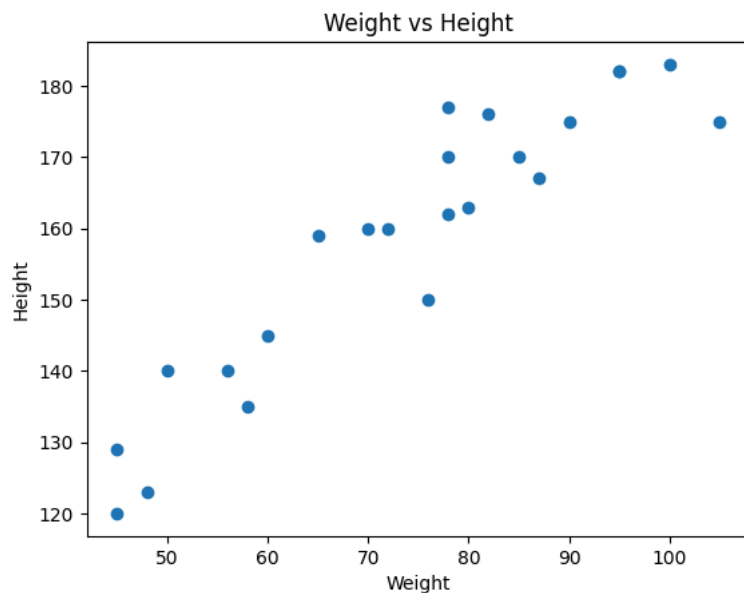
```
(23, 2)
```

```
df.isnull().sum()
```

```
0
Weight 0
Height 0

dtype: int64
```

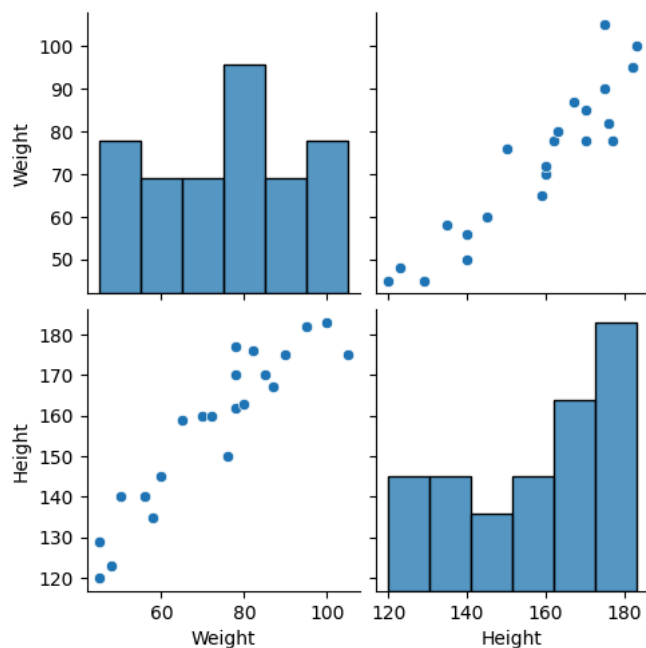
```
plt.scatter(df['Weight'],df['Height'])
plt.xlabel('Weight')
plt.ylabel('Height')
plt.title('Weight vs Height')
plt.show()
```



```
# finding correlation
df.corr()
```

	Weight	Height
Weight	1.000000	0.931142
Height	0.931142	1.000000

```
# seaborn for visualisation
import seaborn as sns
sns.pairplot(df)
plt.show()
```



```
# independet and dependent feature : independent feature should be in dataframe or 2D array dependent can be in a series
x = df['Weight']
y = df['Height']
```

```
x_series = df['Weight']
np.array(x_series).shape
```

```
(23,)
```

```
np.array(y).shape
```

```
(23,)
```

```
# train test split
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.25,random_state=42)
```

```
x_train.shape # 1 feature that has 17 rows
```

```
(17,)
```

```
# standardization
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train.values.reshape(-1, 1))
x_test_scaled = scaler.transform(x_test.values.reshape(-1, 1))
```

```
x_train_scaled
```

```
array([[ -0.87662801],
       [ 1.66773133],
       [ 0.33497168],
       [-1.48242785],
       [ 1.36483141],
       [-1.6641678 ],
       [-0.75546804],
       [-0.1496682 ],
       [ 0.21381171],
       [-1.36126788],
       [-0.99778797],
       [-0.02850823],
       [ 1.06193149],
       [ 0.57729161],
       [ 0.75903157],
       [ 0.88019153],
       [ 0.45613165]])
```

```
x_test_scaled
```

```
array([[ 0.33497168],
       [ 0.33497168],
       [-1.6641678 ],
       [ 1.36483141],
       [-0.45256812],
       [ 1.97063125]])
```

```
# Apply liner regression
from sklearn.linear_model import LinearRegression
regression=LinearRegression(n_jobs=1)
regression.fit(x_train_scaled,y_train)
```

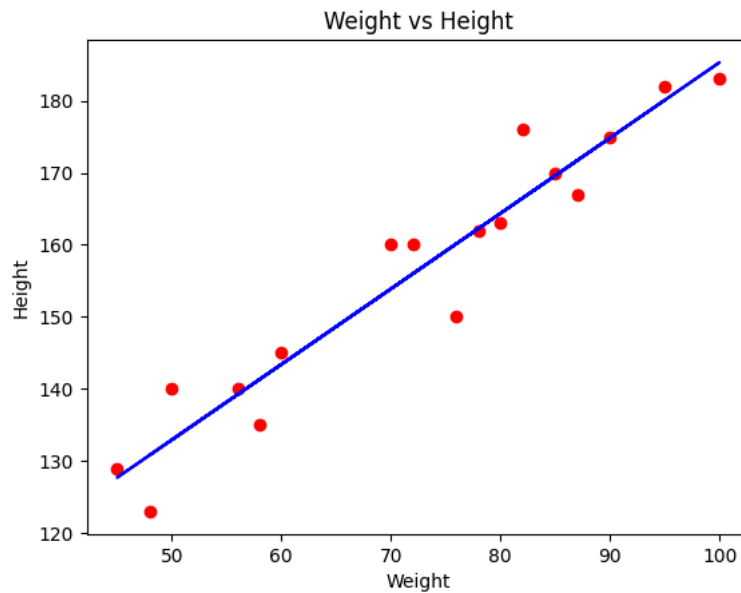
```
▼ LinearRegression ⓘ ?
LinearRegression(n_jobs=1)
```

```
print("cofficient or slope:",regression.coef_)
print("intercept:",regression.intercept_)
```

```
cofficient or slope: [17.2982057]
intercept: 156.47058823529412
```

```
#plot training data
plt.scatter(x_train,y_train,color='red')
plt.plot(x_train,regression.predict(scaler.transform(x_train.values.reshape(-1, 1))),color='blue')
plt.title('Weight vs Height')
plt.xlabel('Weight')
plt.ylabel('Height')
```

Text(0, 0.5, 'Height')



~ # $Y_{pred_test} = intercept + coeff * (x_{test})$

```
# prediction for test data
y_pred_test = regression.predict(x_test_scaled)
```

```
# proformance metrics
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
mes = mean_squared_error(y_test, y_pred_test)
```

```
mes = mean_squared_error(y_test, y_pred_test)
mae = mean_absolute_error(y_test, y_pred_test)
rmse = np.sqrt(mes)
r2 = r2_score(y_test, y_pred_test)
```